IT-414 Software Project Management

Dhirubhai Ambani University
Technology
Formerly DA-IICT

Name	Student ID
Karthik Raman	**202301407**
Bhumsar Boro	**202301002**
Sri Sai Madhava Teja	**202301138**
Neeraj Vania	**202301060**
Lingampalli Venkata Subramanyam	**202301280**
Rishik Yalamanchili	**202301258**

Brief Description of the Project:

The Timetable Generator is a professional software tool built to handle the difficult task of creating college schedules. The project aims to move away from making schedules by hand and instead use a smart system that assigns classes to specific times and rooms. This ensures there are no timing conflicts for students or teachers by following a strict set of rules and university guidelines.

The system works by looking at course data based on the L-T-P-C (Lecture-Tutorial-Practical-Credit) system. It focuses specifically on scheduling the exact number of weekly lectures required by the "L" value of each course.

Key Scheduling Constraints and Priorities:

Constraint/Priority	Description
Core Course Priority	The system will treat mandatory (Core) courses as high priority. It is designed to ensure that no two core courses for a specific student batch overlap, allowing all students in that program to attend their required sessions.
Elective Slot Management	To accommodate elective choices, the system will manage courses grouped into "Slots." While multiple electives within the same slot may be scheduled concurrently, the system will ensure they do not conflict with the Core requirements of the participating batches.
Faculty Fatigue Mitigation	A management constraint considered in the design is the "No Consecutive Lectures" rule. The system will be programmed to prevent any faculty member from being assigned to consecutive teaching periods.
University Parameters	The system will strictly respect the university's parameters such as designated free periods and ensure that room assignments are always

Constraint/Priority	Description
	made based on a capacity that meets or exceeds the student batch strength.

The final result of the project is a complete set of matching schedules. This includes a master weekly plan for student batches, individual timetables for every faculty member, and reports on which rooms are being used. All of these documents are ready to be downloaded in Excel and PDF formats.

Process Model Chosen and Justification

To develop the Timetable Generator, we have chosen the Agile Process Model. This method works step by step, which is perfect for our project as it has many complex rules and specific university requirements that might change as we build it.

Why We Chose Agile

Main Reason	What This Means for the Project
Management of Complex Domain Rules	Instead of trying to figure everything out at once and potentially getting it wrong, we can build the scheduling rules bit by bit and fix things as we go.
Adaptability to Evolving Requirements	If university administration introduces new mandates mid way these can be integrated into the system without requirement of a complete redesign.

Main Reason	What This Means for the Project
Incremental Constraint Validation	We can make sure the absolute "must haves" (like no room double bookings) are solid before we worry about the "nice to have".
Stakeholder Feedbacks	Since we need to make sure the schedules look good for everyone, we can show early versions to the admins and change things based on what they actually want.
Risk Mitigation in Algorithmic Design	Creating a conflict free schedule is hard. Agile gives us the freedom to tweak how the program thinks if it gets stuck on a really tricky scheduling overlap.

Function Point Analysis

To figure out how big this project is, we're using Function Point Analysis (FPA) to look at what the Timetable Generator actually does and how complex the data is. We've broken things down into three categories: "Optimistic" for the easy stuff, "Likely" for the average tasks, and "Pessimistic" for the really tough parts.

Here is how we're scoring them:

- **Optimistic (Simple):** 1 – 3
- **Likely (Average):** 4 – 6
- **Pessimistic (Worst Case):** 7-10

We then get our **Estimated Count (Est. Count)** by plugging those numbers into the PERT formula: (**Optimistic** + 4* **Likely** + **Pessimistic**) / 6.

1. Number of External Inputs (EI)

These represent data entering the system boundary to be processed or to maintain internal logical files .

Sl.No	Details	Optimistic	Likely	Pessimistic	Est. Count	Weight	FP Count
1	Course Details Entry	1	1	2	1.17	4	4.68
2	Faculty Details Entry	1	1	2	1.17	4	4.68
3	Batch Entry	1	1	2	1.17	3	3.51
4	Room Entry	1	1	2	1.17	3	3.51
5	Time-Slot Configuration	1	2	3	2.00	4	8.00
6	Constraint Parameters Entry	2	3	4	3.00	6	18.00
Total							**42.38**

2. Number of External Outputs (EO)

These are the data sets or reports generated by the system for the user .

Sl.No	Details	Optimistic	Likely	Pessimistic	Est. Count	Weight	FP Count
1	Generate Master Timetable	3	4	7	4.34	7	30.38
2	Batch Timetable View	1	1	2	1.17	5	5.85
3	Faculty Timetable View	1	1	2	1.17	5	5.85
4	Room Allocation Schedule	2	2	3	2.17	5	10.85
5	Unresolved Conflict Report	2	3	5	3.17	7	22.19
Total							**75.12**

3. Number of External Enquiries (EQ)

These are interactive tasks that retrieve data from the system without altering its state.

Sl.No	Details	Optimistic	Likely	Pessimistic	Est. Count	Weight	FP Count
1	View Timetable by Batch	1	1	2	1.17	3	3.51
2	View Timetable by Faculty	1	1	2	1.17	3	3.51
3	Check Room Availability	1	1	2	1.17	4	4.68
4	Validate Constraint Input	1	2	3	2.00	3	6.00
Total							**17.70**

4. Number of Internal Logical Files (ILF)

These are the groups of data maintained internally to support the scheduling logic .

Sl.No	Details	Optimistic	Likely	Pessimistic	Est. Count	Weight	FP Count
1	Teacher (Faculty) Table	1	1	2	1.17	7	8.19
2	Course Table	1	1	2	1.17	7	8.19
3	Batch Table	1	1	2	1.17	7	8.19
4	Room Table	1	1	2	1.17	7	8.19
5	TimeSlot Table	1	1	2	1.17	7	8.19
6	Teacher_Course (Assignment)	1	1	1	1.00	7	7.00
7	CourseBatch (Enrollment)	1	1	1	1.00	7	7.00
8	Timetable (Final Schedule)	1	1	2	1.17	7	8.19

9	Constraint Config Table	1	1	1	1.00	4	4.00
Total							**67.14**

5.UFP Summary

The Unadjusted Function Points (UFP) aggregate the totals from all functional components.

Component	**Formula**	**FP Count**
External Inputs (EI)	6 inputs	42.38
External Outputs (EO)	5 outputs	75.12
External Enquiries (EQ)	4 enquiries	17.70
Internal Logical Files (ILF)	9 tables	67.14
External Interface Files (EIF)	0 files	0.00
Total Unadjusted Function Points (UFP)		**202.34**

6.Value Adjustment Factor (VAF)

Evaluated against 14 General System Characteristics (GSCs) on a scale of 0 to 5.

Sl NO.	**General System Characteristic (GSC)**	**Score (0-5)**	**Justification**
1	Data Communications	3	Web/Local client-server interface.
2	Distributed Data Processing	1	Single-server deployment.
3	Performance	3	Algorithms must respond within acceptable time.
4	Heavily Used Configuration	1	Standard university server; no special HW needs.

5	Transaction Rate	2	Input transactions are mostly batch-oriented.
6	Online Data Entry	4	All registry data entered via online forms.
7	End-User Efficiency	4	UI designed for non-technical admin staff.
8	Online Update	3	Admin can update rooms/faculty after setup.
9	**Complex Processing**	**5**	**Highly complex CSP backtracking logic.**
10	Reusability	2	The core algorithm is university-specific.
11	Installation Ease	3	Needs database setup and schema mapping.
12	Operational Ease	3	Conflict reports ease operational recovery.
13	Multiple Sites	1	Deployed at a single institution.
14	Facilitate Change	3	Constraint parameters are admin-configurable.
Sum of all GSC scores		**38**	

7.Final Function Point Calculation

- **Degree of Influence (DI) = 38**
- **VAF = 0.65 + (0.01 × 38) = 1.03**
- **Final FP = UFP × VAF = 202.34 × 1.03 = 208.41**

Team Composition and Roles

Member	Primary Role	Contributions

Kartik Raman (202301407)	Scrum master /project lead	Architecture & Database Design Practitioner : Coding - Data Input & Configuration module
Bhumsar Boro (202301002)	Validation and testing	Prepared sprint plans and Designed Gantt chart Practitioner : Coding - Test & Validation module , User manual
Sri Sai Madhava Teja (202301138)	Core Algorithm Developer	Functional point analysis Practitioner : Coding - CSP backtracking
Neeraj Vania (202301060)	Output & Optimization	Risk analysis Practitioner : Coding - Optimization & Validation module
L V. Subramanyam (202301280)	Requirements & Documentation	System requirement analysis Practitioner : Coding - Output Generation & Main Pipeline
Rishik Yalamanchili (202301258)	Conflict Analysis & Data Integrity	Document for program and logic applied Practitioner : Coding - Conflict Detection & CSP Solver module

Annexure I: System Requirements Specification

1. Project Overview and Introduction

This part outlines the comprehensive functional and non-functional requirements for the **Timetable Generator**. The primary objective of this system is to transition from manual scheduling to an automated, intelligent framework that optimizes academic resource allocation while strictly adhering to institutional constraints and faculty preferences.

2. Functional Requirements (FR)

FR1. Data Management & Input Requirements

- **FR1.1** The system shall allow for the ingestion of Excel based faculty registries, including names, unique identifiers, and contact emails.
- **FR1.2** The system shall prevent duplicate entries for faculty, courses, and rooms through unique database constraints.
- **FR1.3** The system shall ingest Excel course catalog data, specifically interpreting the L-T-P-C structure to automatically calculate the required weekly lecture frequency based on the "L" value.
- **FR1.4** The system shall validate that numerical inputs, such as course credits and batch sizes, are always positive values.
- **FR1.5** The system shall maintain student batch data (year, name, size) and support specific program specializations (e.g., ICT, ICT-CS, MNC).
- **FR1.6** The system shall maintain a digital inventory of rooms, including unique room numbers and seating capacities.
- **FR1.7** The system shall support the mapping of courses to elective "Slots" to ensure that concurrent electives do not conflict with core requirements.

FR2. Time-Slot & Configuration Requirements

- **FR2.1** The system shall allow administrators to configure the standard university time-grid (e.g., 8:00 AM – 1:00 PM, Monday–Friday).
- **FR2.2** The system shall support a fixed number of daily time slots and respect "blackout periods" such as university-wide lunch breaks.
- **FR2.3** The system shall automatically block specific administrative windows (e.g., Wednesday morning free slots) from being scheduled.

FR3. Algorithmic Processing & Constraint Requirement

- **FR3.1** The system shall generate timetables using a Constraint Satisfaction Problem (CSP) algorithm with recursive backtracking.
- **FR3.2** Hard Constraints: The system must strictly enforce the following during generation:
    - **FR3.2.1** Prevent teacher conflicts (no faculty member in two places at once).
    - **FR3.2.2** Prevent batch conflicts (no student group double-booked).
    - **FR3.2.3** Prevent room conflicts (no two classes in the same room).
    - **FR3.2.4** Ensure room capacity is strictly greater than or equal to the batch size.
- **FR3.3** Soft Constraints (Optimization): The system shall attempt to optimize the following via a fitness function:
    - **FR3.3.1** Faculty Fatigue Mitigation: Avoid assigning back-to-back lecture periods to the same faculty member.
    - **FR3.3.2** Course Distribution: Limit a specific course to a maximum of one lecture per day per batch.
    - **FR3.3.3** Load Balancing: Enforce a maximum teaching load of 3 different subjects per faculty member.

FR4. Output & Reporting Requirements

- **FR4.1** The system shall generate a synchronized Master Batch Timetable showing the complete weekly schedule for student groups.
- **FR4.2** The system shall produce Faculty-Specific Schedules to assist staff in managing their teaching commitments.
- **FR4.3** The system shall generate Room Utilization Reports for facility management.
- **FR4.4** Unresolved Conflict Report: In case of mathematical deadlock, the system shall produce a diagnostic report identifying the specific constraints (teacher, room, or slot) preventing a conflict-free solution.
- **FR4.5** All outputs shall be exportable in Excel and PDF formats for formal distribution.

3. Non-Functional Requirements (NFR)

NFR1. Performance & Scalability

- **NFR1.1** The scheduling engine shall be optimized to handle increasing volumes of courses and batches without a significant drop in execution speed.
- **NFR1.2** Database queries shall be optimized for efficient data retrieval and storage.

NFR2. Reliability & Integrity

- **NFR2.1** The system shall guarantee a 100% conflict-free result for all defined hard constraints.

- **NFR2.2** The system shall recover gracefully from input errors by providing clear, helpful error messages to the user.

NFR3. Usability & Accessibility

- **NFR3.1** The user interface shall be intuitive, designed for use by administrative staff with minimal technical training.
- **NFR3.2** Final schedules shall be presented in a clean, legible grid format suitable for both digital and printed displays.

NFR4. Security & Maintainability

- **NFR4.1** The system shall validate all user inputs to prevent data corruption or unauthorized database access.
- **NFR4.2** The codebase shall follow industry standards for documentation and modularity to ensure the system is easy to maintain or update.

4. Operational Constraints

- Scheduling operations are limited to the official university working calendar and operational hours.
- The current scope focuses on the automation of morning lecture sessions; laboratory and practical blocks are managed as independent scheduling units.

System Design Document

Annexure – II: Data Definition

The system will utilize several interconnected data entities to manage the complexity of the scheduling environment.

1. Course Entity

This entity stores the academic details of each subject offered during the semester.

- **Course Code:** A unique alphanumeric identifier (e.g., IT414).
- **Course Title:** The full name of the course.
- **L-T-P-C Structure:** A multi part numeric field representing Lecture hours, Tutorial hours, Practical hours, and total Credits. The "L" value is the primary driver for weekly scheduling frequency .

- **Course Type:** Classification of the course (e.g., Core, Technical Elective, HASS Elective).
- **Assigned Slot:** The pre-determined elective group (e.g., Slot-1, Slot-2) to which the course belongs.

2. Faculty Entity

This entity maintains records for all teaching staff involved in the semester.

- **Faculty ID/Short Name:** A unique code used for internal mapping and timetable display (e.g., HSJ, ST).
- **Full Name:** The complete name of the faculty member.
- **Department/Specialization:** The academic group to which the faculty belongs.

3. Infrastructure (Room) Entity

This entity manages the physical resources available for classes.

- **Room Number/Name:** The unique identifier for a classroom or laboratory.
- **Capacity:** The maximum number of students the room can accommodate. This is used to validate assignments against batch strength.
- **Room Type:** Categorization as a Lecture Hall, Tutorial Room, or Specialized Lab.

4. Student Batch & Section Entity

This entity defines the groups of students for whom the timetable is generated.

- **Program/Batch Name:** The academic program and year (e.g., B.Tech ICT Sem IV).
- **Section:** The specific subgroup (e.g., Sec A, Sec B) within a batch.
- **Batch Strength:** The total number of students enrolled in the specific batch or section.
- **Sub-Batch** : A further subdivision within a program to distinguish specialized streams (e.g., ICT-CS, MNC) that may follow a different course structure within the same semester.
- **Year :** The current academic year of the batch (e.g., 2, 3), used to differentiate cohorts across multiple years who may share the same program name.

5. Timetable Schedule Entity

The central entity that maps all other data points into a finalized schedule.

- **Timetable ID:** A unique primary key identifying each scheduled entry.
- **Assignment ID:** The specific 50-minute block assigned for the session.
- **Batch ID:** Reference to the Student_Batch Entity.
- **Room ID::** Reference to the Room Entity.
- **Slot ID:** Reference to the Time_Matrix Entity.

6. Time Matrix Entity

This entity standardizes the university's weekly scheduling grid, converting all available teaching periods into discrete, referenceable slots.

- **Slot ID:** A unique identifier for each time block in the university week.
- **Day of Week:** The specific day this slot falls on (Monday through Friday).
- **Start Time:** The beginning time of the 50-minute teaching period.
- **End Time:** The concluding time of the teaching period.
- **Slot Group:** A classification label (e.g., Slot-1, Slot-Free) that identifies blackout periods such as Wednesday free slots or lunch breaks, preventing the scheduler from assigning classes to these times.

7. Faculty Course Map Entity

This is a junction entity that resolves the Many-to-Many relationship between Faculty and Courses.

- **Assignment ID:** A unique surrogate primary key generated for each authorized pairing.
- **Faculty ID:** Foreign key referencing the Faculty entity.
- **Course ID:** Foreign key referencing the Course entity.
- **Composite Unique Constraint:** UNIQUE(faculty_id, course_id) ensures the same pairing cannot be registered twice.

8. Batch Course Map Entity

This is a junction entity that resolves the Many-to-Many relationship between Student Batches and Courses.

- **Course ID:** Foreign key referencing the Course entity.
- **Batch ID:** Foreign key referencing the Student Batch entity.
- **Composite Primary Key:** The combination of (course_id, batch_id) together serves as the primary key, ensuring no duplicate enrollment entries exist.

Annexure II:<u>Forms and Tables Interface</u>

The user interface for the **Timetable Generator** is built to prioritize administrative clarity and ease of use. It focuses on simplifying complex data entry while producing highly readable, multi dimensional schedules that meet the needs of students, faculty, and campus coordinators alike.

1. Administrative Input Experience

The system features a streamlined portal for data ingestion, ensuring that setting up a new semester is a smooth and error-free process.

- **Centralized Upload Hub:** A user friendly interface where administrators can easily submit Excel workbooks containing faculty lists, course catalogs, and classroom inventories.
- **Pre Scheduling Check:** Scans for missing details such as unassigned instructors to ensure the scheduling engine has everything it needs to succeed.

2. Output Table: Master Batch Timetable

This grid serves as the essential weekly guide for student cohorts, designed for maximum legibility.

- **Visual Layout:** A clean, matrix mapping specific time periods against the days of the week.
- **Informative Cells:** At a glance, users can see the course code, title, professor, and the specific classroom assigned for each session.
- **Sectional Clarity:** Automatically generates distinct, clutter free timetables for every student batch and academic section.

3. Output Table: Faculty Wise Timetable

A dedicated view that helps professors manage their professional commitments and preparation time effectively.

- **Individualized View:** Each faculty member receives a customized schedule focusing solely on their assigned lectures and student groups.
- **Wellbeing Integration:** The system visually highlights breaks and gaps, ensuring the "No Consecutive Lectures" rule is respected to prevent burnout.

4. Output Table: Room Allocation Matrix

A comprehensive oversight tool for facility managers to track and optimize classroom use.

- **Utilization Tracker:** A master grid showing every room alongside its occupancy status throughout the day.
- **Infrastructure Alignment:** Each entry cross references batch sizes with maximum room capacity to ensure everyone has a comfortable learning space.

5. Conflict and Violation Report

In the rare event that a perfect solution isn't mathematically possible, the system provides a clear path forward.

- **Diagnostic Reporting:** Instead of a simple error, the system generates a detailed breakdown of specific bottlenecks, such as overlapping faculty availability or core course conflicts, allowing for manual adjustment.

Architecture & Database Design

The Timetable Generator relies on a relational database architecture designed in Third Normal Form (3NF). Structuring the data this way isolates independent entities, like faculty, courses, and rooms, into their own tables, which eliminates data redundancy and prevents update anomalies. By offloading basic referential integrity to the database schema, the system guarantees that the Constraint Satisfaction Problem (CSP) solver is always working with valid, logically sound data before the search algorithm even begins.

1. Core Entity Schemas

The database is partitioned into several interconnected tables to accurately map the university's academic environment.

- **Faculty Table:** Maintains unique identifiers, names, and departmental affiliations for all teaching staff.
- **Course Table:** Stores subject metadata, including unique course codes, titles, and the required L-T-P-C credit structure.
- **Student_Batch Table:** Defines the student cohorts. To handle complex institutional structures, this table records the program name, specific sub-batch, section label (e.g., Sec A, Sec B), and the total headcount of the cohort.
- **Room Table:** Catalogs physical infrastructure, storing unique room numbers, facility types, and maximum seating capacities.
- **Time_Matrix Table:** Standardizes the university's scheduling grid, defining distinct time blocks (e.g., Slot-1, Slot-Free) alongside their corresponding days and hourly periods.

2. Relational Mapping & Junction Tables

Because courses are taught to multiple batches and often shared by multiple professors, the architecture utilizes junction tables to manage Many-to-Many relationships. This prevents redundant data entry and creates strict operational boundaries.

- **Batch_Course_Map:** A junction table linking a batch_id to a course_id, explicitly defining which student cohorts are required to take which subjects.
- **Faculty_Course_Map (The Assignment Trick):** Rather than allowing the scheduling algorithm to arbitrarily pair professors and subjects, this junction table explicitly authorizes who can teach what. It combines a faculty_id and a course_id, and generates a unique, surrogate primary key called assignment_id.

3. The Central Fact Table: Master_Timetable

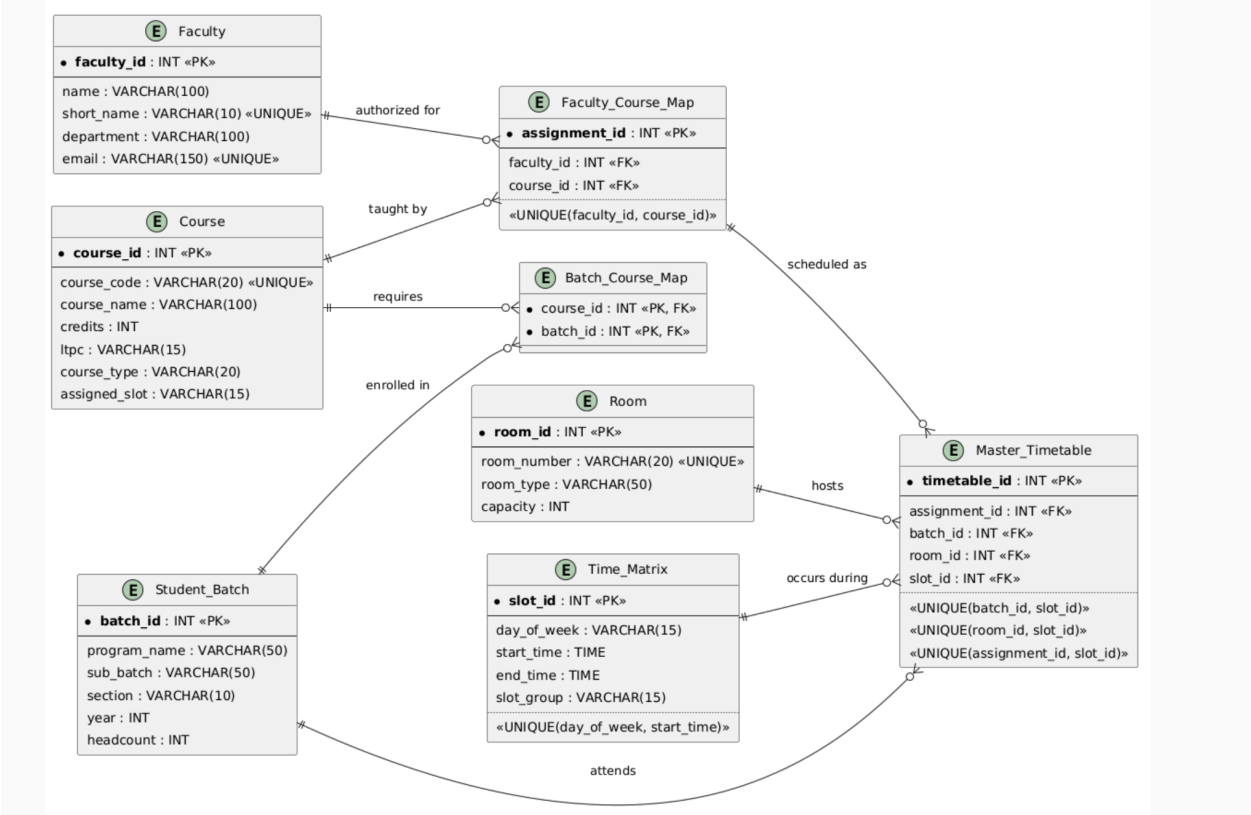
This is the transaction table where the final, generated schedule is stored. Instead of containing raw text, it exclusively relies on Foreign Keys (FK) to link the distinct entities together.

- **Columns:** timetable_id (PK), assignment_id (FK), batch_id (FK), room_id (FK), and slot_id (FK).
- Notice that the final timetable does not reference a faculty_id or course_id directly; it only references the assignment_id. This architectural choice forces the schedule to inherit pre-approved pairings, making it structurally impossible to schedule a professor for a class they are not authorized to teach.

4. Data Integrity and Constraint Enforcement

To ensure absolute scheduling accuracy, the system delegates different types of validation to either the Database Schema or the Application Layer based on computational efficiency.

- **Database-Level Constraints (Hard Locking):** The schema utilizes composite UNIQUE constraints. For example, applying a UNIQUE(room_id, slot_id) constraint ensures that the database engine will physically reject any attempt to insert two different classes into the same room at the same time. This provides a hardware-level guarantee against double-booking.
- **Application-Layer Validation (Dynamic Logic):** Constraints that require comparative logic are handled by the CSP solver before writing to the database. For instance, before proposing a room assignment, the algorithm queries the database to ensure Room.capacity >= Student_Batch.size. Handling this in the application layer keeps the database lightweight and prevents the need for complex, slow-running internal triggers.



Sprint Documentation

The project follows an Agile methodology structured into four sprints from February 16 to April 30, 2025. The development process ensures a systematic progression from requirement analysis to system design, followed by algorithm implementation and optimization.

SPRINT 1: Foundation & Planning | Feb 16 – Mar 7, 2026

Sprint Goal:

Establish the project foundation by gathering requirements, identifying constraints, and defining the overall system scope.

Key Activities:

- Gather and document all university parameters (faculty, courses, rooms, batches)
- Define hard constraints (no double booking, capacity constraints)
- Identify soft constraints (faculty fatigue, scheduling gaps)
- Define system scope and functional requirements
- Prepare initial project plan and architecture outline

Sprint Deliverables:

- System Requirements Specification (SRS)
- Defined hard and soft constraints
- Initial system architecture overview
- Project roadmap and sprint plan

SPRINT 2: Architecture & Database Design | Mar 8 – Mar 28, 2026

Sprint Goal:

Design the complete data architecture by first modeling the system using an Entity-Relationship (ER) diagram and then converting it into a normalized relational schema.

Key Activities

- Design Entity-Relationship (ER) model representing all entities and relationships
- Apply forward-fill and regex techniques for handling inconsistent data
- Identify entities: Faculty, Course, Room, Student_Batch, Time_Matrix
- Convert ER model into relational schema (tables with PK/FK)
- Apply 3NF normalization to eliminate redundancy and ensure data integrity
- Design Excel templates for structured data input
- Implement initial data validation logic

Sprint Deliverables:

- ER Diagram (conceptual design)
- Relational schema with primary and foreign keys
- Fully normalized database schema (3NF)
- Data input templates and validation module

SPRINT 3: Core Scheduling Algorithm (CSP Engine) | Mar 29 – Apr 13, 2026

Sprint Goal:

Develop the core scheduling engine capable of generating conflict-free timetables using constraint-based techniques.

Key Activities:

- Implement Union-Find algorithm for Superblock creation
- Model scheduling as a Constraint Satisfaction Problem (CSP)
- Develop backtracking algorithm
- Integrate Minimum Remaining Values (MRV) heuristic
- Prevent infeasible solutions and optimize search efficiency

Sprint Deliverables:

- Working CSP-based timetable generator
- Conflict-free schedule satisfying all hard constraints
- Core scheduling engine implementation

SPRINT 4: Output Generation, Optimization & Testing | Apr 14 – Apr 30, 2026

Sprint Goal:

Generate user-facing outputs, optimize schedules using soft constraints, and ensure system reliability through testing.

Key Activities:

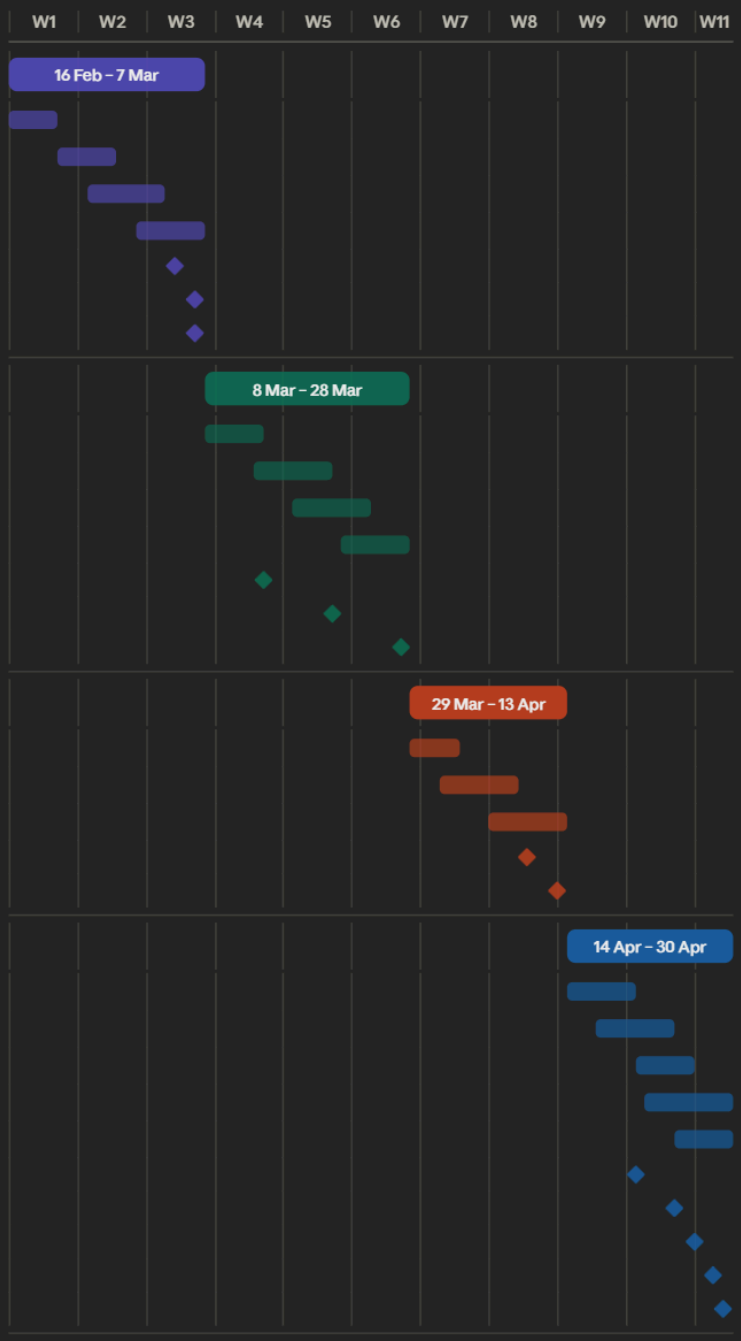
- Generate multiple timetable views:
    - Master timetable
    - Faculty schedules
    - Room allocation
- Implement soft constraint optimization (local search)
- Develop Conflict Audit Report
- Perform system-wide testing and validation
- Optimize performance for scalability
- Prepare User Manual documenting (input format, execution steps, output interpretation).

Sprint Deliverables

- Multi-view timetable outputs (Excel/PDF)
- Soft constraint optimized schedules
- Conflict/diagnostic reports
- Fully tested and deployment-ready system
- User Manual and final project documentation

Gantt Chart:

Feb 16, 2026 – Apr 30, 2026 · 4 Sprints · 11 Weeks



Sprint 1 · Foundation & Planning

- Gather university parameters
- Define hard & soft constraints
- System scope & requirements
- Architecture outline
- ◆ *SRS & hard/soft constraints*
- ◆ *System architecture overview*
- ◆ *Project roadmap & sprint plan*

Sprint 2 · Architecture & Database Design

- ER model design
- Relational schema + 3NF
- Data input templates (Excel)
- Data validation logic
- ◆ *ER diagram*
- ◆ *Normalized schema (3NF)*
- ◆ *Templates & validation module*

Sprint 3 · CSP Scheduling Engine

- Union-Find / Superblock creation
- CSP model & backtracking
- MRV heuristic integration
- ◆ *CSP-based timetable generator*
- ◆ *Conflict-free schedule*

Sprint 4 · Output, Optimization & Testing

- Multi-view timetable outputs
- Soft constraint optimization
- Conflict audit report
- System testing & validation
- User manual & documentation
- ◆ *Multi-view outputs (Excel/PDF)*
- ◆ *Optimized schedules*
- ◆ *Conflict/diagnostic reports*
- ◆ *Deployment-ready system*
- ◆ *User manual & documentation*

Sprint 1 · Foundation & Planning · Sprint 2 · Architecture & Database Design · Sprint 3 · CSP Scheduling Engine · Sprint 4 · Output, Optimization & Testing · ◆ Deliverable milestone

Programs and the logics applied

The Timetable Generator relies on a blend of graph theory, constraint programming, and heuristic search algorithms to solve the complex scheduling problem. The system's logic is structured around several distinct algorithmic phases to handle university constraints.

Data Preprocessing and Entity Resolution Before scheduling begins, the system must translate raw institutional data into structured entities. The logic utilizes forward-filling algorithms to resolve merged cells in spreadsheet inputs, alongside regular expressions to dynamically extract student section labels embedded within course names. This ensures the solver receives clean, discrete entities rather than fragmented text fields.

Conflict Graph Generation (Union-Find) Instead of scheduling individual courses blindly, the system uses a Union-Find (Disjoint Set) algorithm to group related courses into "Superblocks." The mechanism links entries if they share a specific time requirement and an overlapping student group. For example, if a professor is teaching two different lab sections that must occur simultaneously in a shared facility, the algorithm merges them into a single scheduling node. This reduces the overall number of scheduling variables, making the problem computationally manageable.

Once Superblocks are formed, the system builds a conflict graph. Edges are drawn between nodes that share the same faculty member, classroom, or core student batch, which explicitly defines the boundaries for valid schedules.

The Constraint Satisfaction Problem (CSP) The core scheduling engine is formulated as a Constraint Satisfaction Problem. The Superblocks act as the variables, the available time slots defined by the university matrix act as the domain, and the edges in the conflict graph act as the hard constraints. Framing the problem mathematically allows the system to systematically navigate the schedule and avoid double-booking.

Search Strategy: Backtracking with MRV Heuristic To solve the CSP, the algorithm applies a recursive backtracking search. It attempts to assign time slots to Superblocks and "backtracks" to try the next available slot if a constraint is violated. Because naive backtracking has an exponential time complexity, the logic incorporates the Minimum Remaining Values (MRV) heuristic. This heuristic sorts the nodes by their degree of conflict, ensuring that the Superblocks with the most conflict edges are assigned first. By tackling the hardest classes early, the branching factor of the decision tree is drastically reduced. This prevents the solver from exploring dead-end paths, ensuring execution completes rapidly rather than getting stuck in infinite search loops.

Soft Constraint Optimization Once a conflict-free schedule is mathematically secured, the system applies a stochastic local search to satisfy human-centric soft constraints. The optimization mechanism randomly selects pairs of scheduled Superblocks and attempts to swap their time slots. A scoring function then evaluates the new schedule, applying penalty points for undesirable outcomes like scheduling the same course on consecutive days or forcing a student batch to change rooms frequently. If the swap improves the overall score, the new state is kept. This localized refinement polishes the timetable, prioritizing human comfort without breaking the established hard constraints.

Risk Information Sheets

Risk information sheet		
Risk ID: RIS-01	**Prob:** 25%	**Impact:** medium

Description:

The CSP (Constraint Satisfaction Problem) backtracking algorithm may fail to find a conflict-free schedule within acceptable time limits when dealing with large-scale datasets (many batches, courses, faculty, and rooms). The exponential time complexity of naive backtracking could cause the scheduling engine to become computationally stalled, even with the MRV heuristic applied.

Refinement/context:

Subcondition 1: Even though MRV chooses the most difficult classes first, sometimes there are too many conflicts, so the system still has too many possibilities to check, making it slow.

Subcondition 2: The Union-Find algorithm groups related courses into Superblocks, but if the conflict graph becomes too dense (too many edges between nodes), the solver may enter infeasible branches repeatedly.

Mitigation:

1. Introduce a timeout mechanism for the backtracking solver. If no solution is found within a defined time limit, the system should provide a partial schedule with a diagnostic report.
2. Test the system during Sprint 3 using realistic and realistic datasets.
3. Design the system to output an Unresolved Conflict Report (Conflict Audit Report) when deadlocks occur, providing administrators with actionable bottleneck details.

Management/contingency plan/trigger:

Allocate buffer time in Sprints 3 and 4 for algorithmic refinement. If initial tests during Sprint 3 reveal execution time exceeding time for standard datasets, escalate to the Algorithm & Scheduling Team and consider switching to a more efficient solver.

Trigger: Solver execution exceeds timeout threshold during Sprint 3 testing.

Current status:

Pre-development phase.

Originator: Sri Sai	**Assigned:** Sri sai & Rishik

Risk information sheet		
Risk ID: RIS-02	**Prob:** 40%	**Impact:** medium

Description:

The input data quality from Excel workbook uploads may be inconsistent, containing merged cells, missing faculty assignments, incomplete L-T-P-C structures, or ambiguous section labels embedded within course names. Poor data quality will directly compromise the scheduling program's ability to produce valid, conflict-free timetables.

Refinement/context:

Subcondition 1: University administrative data is often maintained manually in Excel spreadsheets. Merged cells and irregular formatting are common.

Subcondition 2: The system expects structured data, but administrators may submit workbooks that deviate from the expected template format (e.g., missing columns, extra sheets, inconsistent naming conventions).

Subcondition 3: Unassigned instructors or courses with incomplete metadata (e.g., missing room type requirements or batch strength) may pass through initial validation and only cause failures during the scheduling phase.

Mitigation:

1. Implement robust Pre-Scheduling Checks in the input validation to scan for missing details such as unassigned instructors, incomplete L-T-P-C values, and mismatched batch sizes before the scheduling program runs.
2. Include sample/test data files for administrators to use as reference during data entry.

Management/contingency plan/trigger:

Prioritize the input validation module in Sprint 2. If more than 20% of test data uploads fail validation during Sprint 2 testing, allocate additional development resources.

Trigger: High failure rate in data ingestion during Sprint 2 integration testing.

Current status:

Pre-development phase.

Originator: Karthik	**Assigned:** Karthik

Risk information sheet		
Risk ID: RIS-03	**Prob:** 30%	**Impact:** medium-high

Description:

The soft constraint optimization phase (stochastic local search) may produce schedules that, while technically conflict-free on hard constraints, result in poor faculty experience, such as excessive gaps between lectures, courses scheduled on consecutive days, or students frequently changing rooms. This can lead to stakeholder dissatisfaction despite technical correctness.

Refinement/context:

Subcondition 1: The "No Consecutive Lectures" rule for faculty is classified as a soft constraint and is handled post-scheduling. If the hard-constraint solution is very tight, there may be limited room for soft constraint improvements.

Subcondition 2: The scoring function for stochastic optimization needs careful calibration. Incorrect penalty weights may prioritize some soft constraints (e.g., room changes) over more important ones (e.g., faculty fatigue).

Subcondition 3: Stakeholder preferences may evolve during development. University administration may introduce new soft constraints or reprioritize existing ones mid-project.

Mitigation:

1. Conduct stakeholder feedback sessions at the end of Sprint 3, presenting sample schedules to administrators and faculty for review.
2. Implement visual highlighting of breaks and gaps in the Faculty Wise Timetable to make it easy to verify soft constraint adherence.
3. Maintain a log of all soft constraint satisfaction rates per generated schedule for trend analysis.

Management/contingency plan/trigger:

Leverage the Agile model's built-in adaptability. If stakeholder feedback during Sprint 3 indicates dissatisfaction with schedule quality, allocate Sprint 4 time for scoring function refinement.

Trigger: More than 30% of faculty schedules violate the "No Consecutive Lectures" rule after soft constraint optimization.

Current status:

Pre-development phase.

Originator: Neeraj	**Assigned:** Neeraj & venkat

Risk information sheet

Risk ID: RIS-04	**Prob:** 25%	**Impact:** high

Description:

The 3NF (Third Normal Form) database schema design relies on composite UNIQUE constraints and Foreign Key relationships to enforce hard constraints at the database level. If the schema is not correctly implemented, it may allow double-bookings (same room at same time, same faculty at same time) to be written to the Master_Timetable, fundamentally breaking scheduling integrity.

Refinement/context:

Subcondition 1: The UNIQUE(room_id, slot_id) constraint must be correctly implemented to prevent two classes from being assigned the same room during the same time period. Any gap in this constraint allows physical double-booking.

Subcondition 2: If the mapping between teacher and course is wrong, the system may assign a teacher to a subject they are not authorized to teach.

Subcondition 3: The separation of constraint enforcement between Database-Level and Application-Layer requires both layers to be perfectly synchronized. A failure in either layer could silently allow invalid schedules.

Mitigation:

1. Develop tests during Sprint 2 that explicitly attempt to insert duplicate room-slot and faculty-slot combinations to verify that database constraints correctly reject them.
2. Test the system from start to finish to ensure that when a professor is paired with a course, that exact pairing makes it all the way to the final Master Timetable without any data getting mixed up along the way.
3. Conduct a formal schema review with the full team before Sprint 3 begins, walking through the ER model and verifying all constraints.

Management/contingency plan/trigger:

Assign the Testing & database management team to develop a comprehensive constraint test suite during Sprint 2. If any database-level constraint test fails, halt Sprint 3 development until the schema is corrected.

Trigger: Any test case that successfully inserts a double-booked entry into the database.

Current status:

Pre-development phase.

Originator: Karthik	**Assigned:** Karthik & Boro

Risk information sheet		
Risk ID: RIS-05	**Prob:** 40%	**Impact:** medium-high

Description:

The team consists of only 6 members with specific role allocations. If any team member becomes unavailable (due to academic workload, personal reasons, or other commitments), the project timeline may be severely impacted.

Refinement/context:

Subcondition 1: The Algorithm & Scheduling Team is responsible for the core scheduling engine, which is the most technically demanding and critical component. Loss of even one member would affect the team's capacity for the most complex work.

Subcondition 2: As students, team members have competing academic responsibilities that may conflict with sprint deadlines, especially during examination periods.

Mitigation:

1. Conduct cross-training sessions so that at least one other team member has basic familiarity with each critical component, especially the CSP solver logic.
2. Maintain thorough code documentation and design documents to enable knowledge transfer if a member becomes unavailable.
3. Schedule sprints to avoid overlap with university examination periods where possible.

Management/contingency plan/trigger:

If a team member becomes unavailable for more than 3 working days during an active sprint, the Scrum Master should redistribute tasks and adjust sprint scope.

Trigger: Team member absence exceeding 3 consecutive days during a sprint.

Current status:

Pre-development phase.

Originator: Karthik	**Assigned:** All team members

Risk information sheet

Risk ID: RIS-06	**Prob:** 30%	**Impact:** medium

Description:

The system must export schedules in both Excel and PDF formats for formal distribution. The multi-view output generation (Master Batch Timetables, Faculty Schedules, Room Allocation Reports) requires complex formatting in clean, legible grid layouts. There is a risk that the generated outputs may have formatting issues, incorrect data mapping, or poor readability across different devices and print formats.

Refinement/context:

Subcondition 1: The Master Batch Timetable requires a matrix format mapping time periods against days of the week, with each cell containing course code, title, professor name, and room. Complex cell content may overflow or truncate in Excel/PDF rendering.

Subcondition 2: Faculty Wise Timetables need individualized views with visual highlighting of breaks and gaps. PDF generation may not preserve all visual formatting consistently across different PDF viewers.

Mitigation:

1. Use well-supported libraries for Excel and PDF generation with thorough formatting templates.
2. Test output rendering on multiple platforms (Windows, Mac) and viewers.

Management/contingency plan/trigger:

Schedule output generation development in Sprint 4 with sufficient buffer time.

Trigger: Feedback during Sprint 4 demo indicates poor readability or data errors in exported files.

Current status:

Pre-development phase.

Originator: Venkat	**Assigned:** Venkat & Neeraj